

[Home](#)[GitHub](#)

CLASSES

[ErrorBoundary](#)

EVENTS

[SIGINT](#)[unhandledRejection](#)

NAMESPACES

[apiConfig](#)[contactoEmergenciaService](#)[diarioService](#)

GLOBAL

[404_Error_Handler](#)[ACTIVIDADES](#)[API_URL](#)[ASPECTOS_COGNITIVOS](#)[ActionCard](#)[ActivitySelector](#)[App](#)[AuthButtons](#)[AuthLayout](#)[Button](#)[CORS_Configuration](#)[Card](#)[Carousel](#)[Checkbox](#)[CircularProgress](#)[CognitionSelector](#)[Diario](#)[DiaryBanner](#)[DiaryEditor](#)[DiaryEntry](#)[EMOCIONES](#)[EmergencyButton](#)[EmergencyModal](#)[EmotionSelector](#)[EmotionStats](#)[Footer](#)[GET /](#)

frontend/src/components/atoms/Textarea.jsx

```
1.  /**
2.   * @file Componente de área de texto reutilizable.
3.   * @description Un componente de textarea con contador de caracteres
4.   * @requires react
5.   * @requires prop-types
6.   * @requires ../../styles/atoms/Textarea.css
7.   */
8.  import React from 'react';
9.  import PropTypes from 'prop-types';
10. import '../../styles/atoms/Textarea.css';
11.
12. /**
13.  * @function Textarea
14.  * @description Renderiza un campo de área de texto con funcionalidad
15.  * @param {object} props - Las propiedades del componente.
16.  * @param {string} props.value - El valor del área de texto.
17.  * @param {function} props.onChange - Función que se ejecuta cuando
18.  * @param {function} [props.onBlur] - Función que se ejecuta cuando
19.  * @param {string} [props.placeholder] - El texto de marcador de pos
20.  * @param {string} [props.name] - El nombre del área de texto.
21.  * @param {boolean} [props.disabled=false] - Si es `true`, el área d
22.  * @param {number} [props.rows=4] - El número de filas visibles del
23.  * @param {number} [props.maxLength] - El número máximo de caractere
24.  * @param {string} [props.className=''] - Clases CSS adicionales par
25.  * @param {string} [props.error] - Un mensaje de error para mostrar
26.  * @returns {JSX.Element} El componente de área de texto.
27.  */
28.  const Textarea = ({
29.    value,
30.    onChange,
31.    onBlur,
32.    placeholder,
33.    name,
34.    disabled = false,
35.    rows = 4,
36.    maxLength,
37.    className = '',
38.    error
39.  }) => {
40.    return (
41.      <div className={`textarea-wrapper ${className}`}>
42.        <textarea
43.          name={name}
44.          value={value}
45.          onChange={onChange}
46.          onBlur={onBlur}
47.          placeholder={placeholder}
48.          disabled={disabled}
49.          rows={rows}
50.          maxLength={maxLength}
51.          className={`textarea ${error ? 'textarea--error' : ''}}
52.        </>
```

```

53.         {maxLength && value && (
54.             <div className="textarea__counter">
55.                 {value.length} / {maxLength}
56.             </div>
57.         )}
58.         {error && <span className="textarea__error">{error}</span>}
59.     </div>
60. );
61. };
62.
63. Textarea.propTypes = {
64.     /** El valor del área de texto. */
65.     value: PropTypes.string,
66.     /** Función que se ejecuta cuando cambia el valor. */
67.     onChange: PropTypes.func.isRequired,
68.     /** Función que se ejecuta cuando el área de texto pierde el foco. */
69.     onBlur: PropTypes.func,
70.     /** El texto de marcador de posición. */
71.     placeholder: PropTypes.string,
72.     /** El nombre del área de texto. */
73.     name: PropTypes.string,
74.     /** Si es `true`, el área de texto está deshabilitada. */
75.     disabled: PropTypes.bool,
76.     /** El número de filas visibles del área de texto. */
77.     rows: PropTypes.number,
78.     /** El número máximo de caracteres permitidos. */
79.     maxLength: PropTypes.number,
80.     /** Clases CSS adicionales para el contenedor. */
81.     className: PropTypes.string,
82.     /** Un mensaje de error para mostrar debajo. */
83.     error: PropTypes.string
84. };
85.
86. export default Textarea;

```